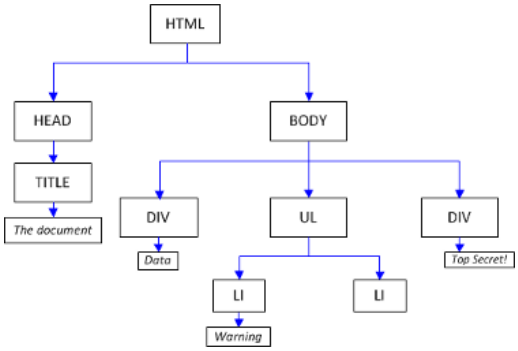
3 - Manipuler les éléments d'une page avec DOM

Comprendre l'arbre DOM

Arbre DOM

Lorsqu'une page Web est chargée, le navigateur crée un Document Objet Modèle de la page.

Le modèle HTML DOM est construit comme un arbre d' Objets :



Avec le modèle objet, JavaScript obtient toute la puissance dont il a besoin pour créer du HTML dynamique :

- JavaScript peut modifier tous les éléments HTML de la page
- JavaScript peut modifier tous les attributs HTML de la page
- JavaScript peut changer tous les styles CSS de la page
- JavaScript peut supprimer les éléments et attributs HTML existants
- JavaScript peut ajouter de nouveaux éléments et attributs HTML
- JavaScript peut réagir à tous les événements HTML existants dans la page
- JavaScript peut créer de nouveaux événements HTML dans la page

Le DOM HTML est un modèle objet standard et une interface de programmation pour HTML. Il définit :

- Les éléments HTML comme objets
- Les propriétés de tous les éléments HTML
- Les méthodes pour accéder à tous les éléments HTML
- Les événements pour tous les éléments HTML
- En d'autres termes : le DOM HTML est une norme sur la façon d'obtenir, de modifier, d'ajouter ou de supprimer des éléments HTML

Objet Document

L'objet document représente votre page Web. Si vous souhaitez accéder à n'importe quel élément d'une page HTML, vous commencez toujours par accéder à l'objet document.

Vous trouverez ci-dessous quelques exemples d'utilisation de l'objet document pour accéder au HTML et le manipuler.

Recherche d'éléments HTML

Method	Description
document.getElementById(id)	Trouver un élément par élément ID
document.getElementsByTagName(name)	Trouver des éléments par nom de balise
document.getElementsByClassName(name)	Trouver des éléments

	par nom de classe

Modification des éléments HTML

Propriété	Description
element.innerHTML = new html content	Changerla valeur de l'innerHTML
element.attribute = new value	Changer l'attribut d'un élément
element.style.property = new style	Changer le style d'un élément

Ajout et suppression d'éléments

Method	Description
document.createElement(element)	Créer un élément HTML
document.removeChild(element)	Supprimer un élément HTML
document.appendChild(element)	Ajouter un élément HTML enfant
document.replaceChild(new, old)	Remplacer un élément HTML
document.write(text)	Ecrire dans un docment HTML

Ajout de gestionnaire d'évènement

Méthode	Description
document.getElementById(id).onclick = function(){code}	Ajouter un événement de clic

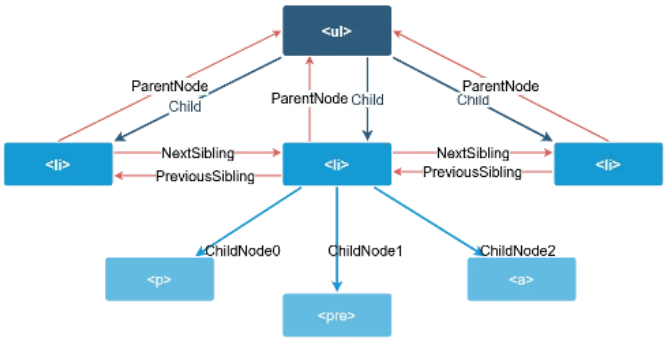
Relations de noeuds

Les nœuds de l'arborescence des nœuds ont une relation hiérarchique les uns avec les autres.

Présentation générale

Les termes parent, enfant et frère sont utilisés pour décrire les relations.

- Dans une arborescence de nœuds, le nœud supérieur est appelé racine (ou nœud racine)
- Chaque nœud a exactement un parent, sauf la racine (qui n'a pas de parent)
- Un nœud peut avoir plusieurs enfants
- Les frères et sœurs (frères ou sœurs) sont des nœuds avec le même parent



Navigation entre les noeuds

1 - Descendre dans l'arborescence DOM : Il existe 6 différentes manières JavaScript de déplacer et d'obtenir des nœuds enfants

- **firstChild**- Retourne le premier enfant de l'élément
- **firstElementChild** - Retourne le premier élément enfant du parent
- **lastChild**- Retourne le dernier enfant de l'élément
- **lastElementChild** - Retourne le premier élément enfant du parent
- **childNodes** - Renvoie tous les enfants de l'élément sous la forme d'une collection de tableaux
- **children** - Renvoie tous les enfants qui sont des éléments sous la forme d'une collection de tableaux

2 - Remonter dans l'arborescence DOM : Il existe 2 manières JavaScript différentes de déplacer et d'accéder aux nœuds parents

- **parentNode** - Renvoie le nœud parent de l'élément
- **parentElement** - Renvoie le nœud de l'élément parent de l'élément

3 - Déplacer latéralement dans l'arborescence : il existe 4 façons différentes de se déplacer et d'accéder aux nœuds frères en JavaScript

- **nextSibling** - Renvoie le nœud frère qui est le prochain enfant de son parent
- **nextElementSibling** - Renvoie l'élément frère qui est le prochain enfant de son parent
- **previousSibling** - Renvoie le nœud frère qui est un enfant précédent de son parent
- **previousElementSibling** - Renvoie l'élément frère qui est un enfant précédent de son parent

Connaître les bases de la manipulation DOM

Sélecteurs (simples, multiples...)

La méthode querySelector() renvoie le premier élément qui correspond à un ou plusieurs sélecteurs CSS spécifiés dans le document.

Remarque : La méthode querySelector() ne renvoie que le premier élément qui correspond aux sélecteurs spécifiés. Pour renvoyer toutes les correspondances, utilisez plutôt la méthode querySelectorAll() .

Si le sélecteur correspond à un identifiant dans un document qui est utilisé plusieurs fois (notez qu'un "identifiant" doit être unique dans une page et ne doit pas être utilisé plus d'une fois), il renvoie le premier élément correspondant.

Syntaxe :

```
document.querySelector(CSS selectors)
```

Exemples

Obtenez le premier élément <p> dans le document :

```
document.querySelector("p");
```

Obtenez le premier élément <p> du document avec class="example" :

```
document.querySelector("p.example");
```

Changez le texte d'un élément avec id="demo" :

```
document.querySelector("#demo").innerHTML = "Hello World!";
```

Obtenez le premier élément <p> dans le document où le parent est un élément :

```
document.querySelector("div > p");
```

Obtenez le premier élément <a> dans le document qui a un attribut "target" :

```
document.querySelector("a[target]");
```

Obtenez tous les éléments <p> du document et définissez la couleur d'arrière-plan du premier élément <p> (index 0) :

```
// Get all <p> elements in the document

var x = document.querySelectorAll("p");
// Set the background color of the first <p> element

x[0].style.backgroundColor = "red";
```

Modes d'accès aux éléments

La façon la plus simple d'accéder à un élément dans un document va être de la faire en le ciblant avec le sélecteur CSS qui lui est associé. Deux méthodes nous permettent de faire cela : les méthodes querySelector() et querySelectorAll() qui sont des méthodes du mixin ParentNode et qu'on va pouvoir implémenter à partir des interfaces Document et Element.

```
/*Sélectionne le premier paragraphe du document et change son texte avec la propriété textContent que
nous étudierons plus tard dans cette partie*/
document.querySelector('p').textContent = '1er paragraphe du document';
let documentDiv = document.querySelector('div'); //1er div du document
//Sélectionne le premier paragraphe du premier div du document et modifie son texte
documentDiv.querySelector('p').textContent = '1er paragraphe du premier div';
/*Sélectionne le premier paragraphe du document avec un attribut class='bleu' et change sa couleur en
bleu avec la propriété style que nous étudierons plus tard dans cette partie*/
document.querySelector('p.bleu').style.color = 'blue';
//Sélectionne tous les paragraphes du document
let documentParas = document.querySelectorAll('p');
//Sélectionne tous les paragraphes du premier div
let divParas = documentDiv.querySelectorAll('p');
/*On utilise forEach() sur notre objet NodeList documentParas pour rajouter du texte dans chaque
paragraphe de notre document*/
documentParas.forEach(function(nom, index){
    nom.textContent += ' (paragraphe n°:' + index + ')';
});
```

Accéder à un élément en fonction de la valeur de son attribut id

La méthode getElementById() est un méthode du mixin NonElementParentNode et qu'on va implémenter à partir d'un objet Document. Cette méthode renvoie un objet Element qui représente l'élément dont la valeur de l'attribut id correspond à la valeur spécifiée en argument.

La méthode getElementById() est un moyen simple d'accéder à un élément en particulier (si celui-ci possède un id) puisque les id sont uniques dans un document.

```
//Sélectionne l'élément avec un id = 'p1' et modifie la couleur du texte
document.getElementById('p1').style.color = 'blue';
```

Accéder à un élément en fonction de la valeur de son attribut class

Les interfaces Element et Document vont toutes deux définir une méthode getElementsByClassName() qui va renvoyer une liste des éléments possédant un attribut class avec la valeur spécifiée en argument. La liste renvoyée est un objet de l'interface HTMLCollection qu'on va pouvoir traiter quasiment comme un tableau.

En utilisant la méthode getElementsByClassName() avec un objet Document, la recherche des éléments se fera dans tout le document. En l'utilisant avec un objet Element, la recherche se fera dans l'élément en question.

```
//Sélectionne les éléments avec une class = 'bleu'
let bleu = document.getElementsByClassName('bleu');
//"bleu" est un objet de HTMLCollection qu'on va manipuler comme un tableau
for(valeur of bleu){
    valeur.style.color = 'blue';
}
```

Manipuler les éléments HTML

Manipulation des éléments (Création, modification, suppression)

Créer un élément en JavaScript

Il est courant que les sites Web interactifs créent de nouveaux éléments dynamiquement sur l'action de l'utilisateur et les utilisent. Alors, comment créer un nouvel élément ?

La façon dont nous créons de nouveaux éléments est d'utiliser la méthode createElement. Appelez la méthode createElement via documentobject et transmettez le nom de la balise de l'élément que vous souhaitez créer.

```
<script>
    function addChild() {
      const parent = document.getElementById("parent"); // selecting
parent
      const child = document.createElement("p"); // creating child
      child.innerHTML = "This is second child"; // adding some content
      parent.append(child);
    }
  </script>
```

La méthode createElement convertit le nom de balise de l'élément en minuscules avant de créer l'élément.

```
const parent = document.getElementById("parent"); // selecting parent
const child = document.getElementById("secondChild"); // selecting child

function remove() {
  parent.removeChild(child);
}
```

Ajouter un élément en JavaScript

Pour que l'élément créé fasse partie du document, nous devons le spécifier comme élément parent et lui ajouter un élément nouvellement créé.

Pour ajouter un élément, utilisez la méthode append(). Il insère un ensemble d' objets Node ou d' objets DOMString en tant que dernier enfant du ParentNode

```
<script>
    function addChild() {
      const parent = document.getElementById("parent"); // selecting
parent
      const child = document.createElement("p"); // creating child
      child.innerHTML = "This is second child"; // adding some content
      parent.append(child);
    }
  </script>
```

Supprimer un élément en JavaScript

Dans une application Web interactive, vous souhaiterez non seulement créer et ajouter un nouvel élément au document, mais également supprimer tout élément souhaité.

La méthode removeChild supprime un élément de la structure DOM. Le nœud à supprimer est passé en argument à la méthode.

La méthode removeChild renvoie une référence au nœud enfant qui est supprimé.

```
const parent = document.getElementById("parent"); // selecting parent
const child = document.getElementById("secondChild"); // selecting child

function remove() {
  parent.removeChild(child);
}
```

Modifier un élément en JavaScript

La replaceChild()méthode remplace un nœud par un autre nœud dans le DOM.

```
<div class="col-md-12">
    <div class="p-3">
        <label>Input String:</label><br>
        <input type="text" class="input-blue-border" id="b1" value="120">
        <button class="ml-3" onclick="changeStyle()">Change Border</button>
    </div>
  </div>
```

```
function changeStyle() {
    document.getElementsByClassName("input-blue-border")[0].style.borderColor = "red";
  }
```

Mise à jour des styles, attributs et classes

Une fois que nous avons identifié l'élément de manière unique, nous pouvons utiliser les méthodes .style ou .className pour changer ses styles CSS.

```
<a href="https://www.google.com/" id="myLink">Google</a>

<script>
    // Selecting the element
    var link = document.getElementById("myLink");

    // Removing the href attribute
    link.removeAttribute("href");
</script>
```

Définir le style à l'aide de element.className

On peut utiliser element.className pour changer divers paramètres de style d'un élément HTML en les regroupant comme une classe et en attribuant le nom de la classe à l'élément sélectionné avec element.className. Cette méthode est utile, en particulier dans les scénarios où nous devons afficher une erreur dans un champ de saisie. Dans ce cas, nous devons changer la couleur de la bordure du champ de saisie en couleur rouge et le texte intérieur de l'entrée en couleur rouge. Par conséquent, nous pouvons regrouper ces styles en tant que classe et les affecter à l'élément en utilisant l'attribut element.className. Le code suivant illustre la gestion des erreurs.

```
<div class="col-md-12">
    <div class="p-3">
        <label>Input String:</label><br>
        <input type="text" class="input-blue-border" id="b1" value="120">
        <button class="ml-3" onclick="changeClass()">Change Border</button>
    </div>
  </div>
```

```
function changeClass() {
    document.getElementsByClassName("input-blue-border")[0].className = "input-error";
  }
```

Mise à jour d'un attribut avec setAttribute

La setAttribute()méthode est utilisée pour définir un attribut sur l'élément spécifié.

Si l'attribut existe déjà sur l'élément, la valeur est mise à jour ; sinon, un nouvel attribut est ajouté avec le nom et la valeur spécifiés. Le code JavaScript dans l'exemple suivant ajoutera les attributs class et disabled à l'élément <button>.

```
<button type="button" id="myBtn">Click Me</button>
<script>
    // Selecting the element
    var btn = document.getElementById("myBtn");

    // Setting new attributes
    btn.setAttribute("class", "click-btn");
    btn.setAttribute("disabled", "");
</script>
```

Suppression d'attributs d'éléments

La méthode removeAttribute() est utilisée pour supprimer un attribut de l'élément spécifié.

Le code JavaScript dans l'exemple suivant supprimera l'attribut href d'un élément d'ancrage.

```
<a href="https://www.google.com/" id="myLink">Google</a>

<script>
    // Selecting the element
    var link = document.getElementById("myLink");

    // Removing the href attribute
    link.removeAttribute("href");
</script>
```

Création DOMMenuObject

Syntaxe

```
var menuObject = document.createElement("MENU")
```

```
<!DOCTYPE html>
<html>
<head>
<title>Menu Object</title>
<link rel="stylesheet" href="style.css">
<script src="script.js"></script>
</head>
<body>
<form contextmenu="MENU">
<label for="urlSelect">Current URL:</label>
<input type="url" size="30" id="urlSelect" value="https://www.example.com/aboutUs">
<div id="divDisplay"></div>
<menu type="context" id="MENU">
<menuitem label="Get URL" onclick="contextFunction(1);"></menuitem>
<menuitem label="Get Hostname" onclick="contextFunction(2);"></menuitem>
</menu>
</form>
</body>
</html>
```

Script.js

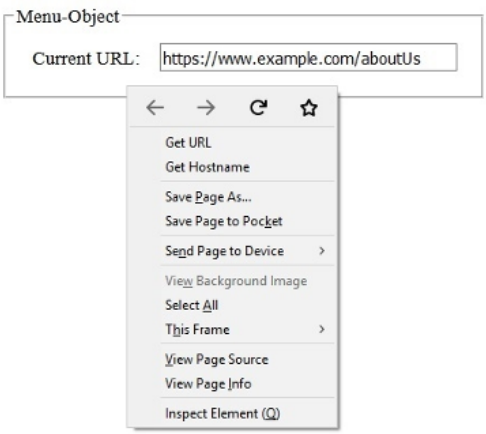
```javascript
var divDisplay = document.getElementById("divDisplay");
  var urlSelect = document.getElementById("urlSelect");
  function gethref(){
     divDisplay.textContent = 'URL Path: '+location.href;
  }
  function getHostname(){
     divDisplay.textContent = 'Hostname: '+location.hostname;
  }
  function contextFunction(role){
     if(role === 1)
        this.gethref();
     else
        this.getHostname();
  }
```

style.js

```css
form {
   width:70%;
   margin: 0 auto;
   text-align: center;
}
* {
   padding: 2px;
   margin:5px;
}
input[type="button"] {
   border-radius: 10px;
}
```

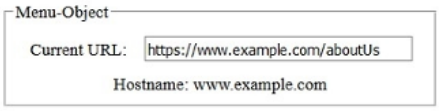
Résultat

Cela produira la sortie suivante :
Clic droit sur le formulaire



Après avoir cliqué sur l' élément de menu "**Obtenir l'URL**" dans le menu contextuel

Après avoir cliqué sur l' élément de menu "**Obtenir le nom d'hôte**" dans le menu contextuel

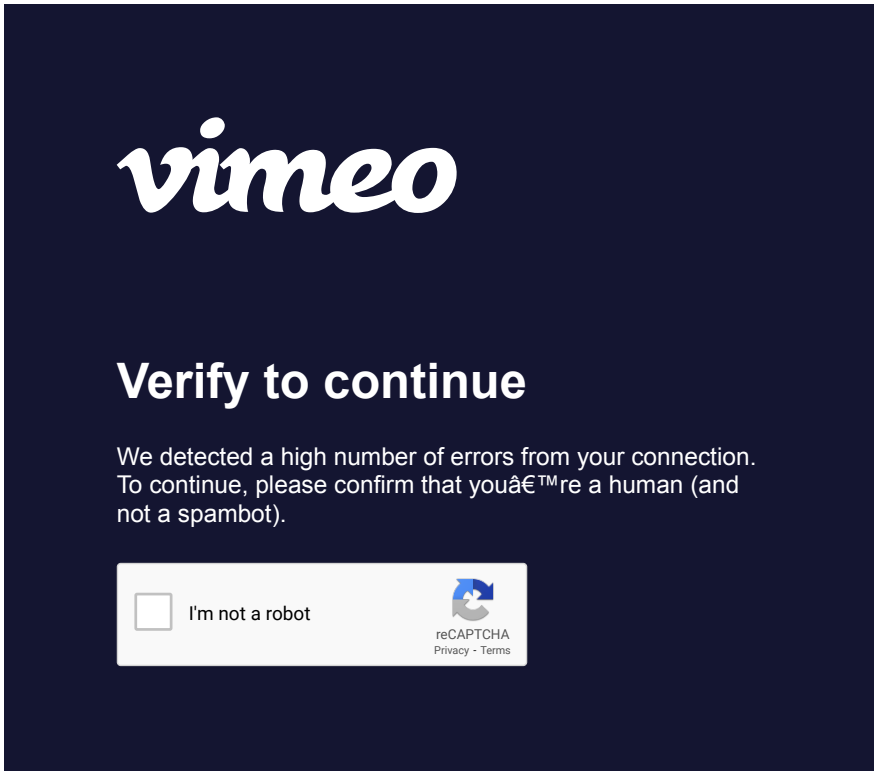


Programmer de la musique avec Javascript

Dans cette vidéo, vous allez apprendre à programmer en Javascript avec une vidéo explicative et ludique. Vous allez suivre pas à pas les explications du formateur et découvrir les mille et une possibilité du langage Javascript.

Vous pouvez télécharger **la fiche PDF " Apprendre Javascript en musique"** de consignes à droite de l'écran.

C'est parti !


vimeo
Verify to continue
We detected a high number of errors from your connection.
To continue, please confirm that youâ€™re a human (and
not a spambot).
I'm not a robot
reCAPTCHA
Privacy - Terms